

Probability theorems simulator and prover

با توجه به اینکه این پروژه 4 فاز دارد، طراحی آن را نیز در 4 بخش انجام دادم.

```
===== Probability theorems simulator and prover =====  
  
1. Phase 0: Comparison of random number generation engines(LCG Vs XorShift)  
2. Phase 1 & 2: Distribution simulation and statistical analysis  
3. Phase 3: Examining the memoryless property(Geometry / Exponential)  
4. Phase 3: Examining Limit Approximation Theorems(Normal instead of Binomial / Poisson)  
5. Exit  
-----  
Your Choice:  █
```

random_number_generator.py:

در این فایل موتور های تولید اعداد تصادفی را پیاده سازی کردم. این ماژول شامل یک کلاس انتزاعی برای تولید اعداد است و دو کلاس دیگر که از آن مشتق شده اند با روش های ذکر شده در توضیحات پروژه عدد تصادفی تولید میکنند.

هر کدام از دو کلاس تولید کننده اعداد فرمول های خاص خود را دارند که برای پیاده سازی این کلاس ها از آن فرمول ها استفاده شده است.

این ماژول یک تابع برای مقایسه این دو موتور نیز دارد که با گرفتن یک ورودی به عنوان تعداد دفعات آزمایش، زمان تولید اعداد را برای این دو موتور مقایسه میکند.

```
0.5890072894231451  
0.9708677446334006  
0.7569011988848919  
0.3777713726171159  
0.40192760080188866
```

```
Time spent in LCG: 0.00032591819763183594
```

```
=====
```

```
0.45924658199289037  
0.40725155254063466  
0.04155395716464937  
0.48145706892978796  
0.5325363312225175
```

```
Time spent in XorShift: 0.0003616809844970703
```

```
Press ENTER to return... █
```

```
Enter Your Number: 5 █
```

distributions.py:

در این ماژول توزیع ها را پیاده سازی کردم؛ یک کلاس انتزاعی برای ساختاردهی به ماژول و استفاده از polymorphism طراحی کردم که هر کدام از توزیع ها به صورت یک زیرکلاس از این کلاس مشتق شده اند.

هر کلاس یک متد generate_sample دارد که با آن توزیع را انجام می دهد. همچنین برای فاز های دیگر پروژه در این ماژول، برای هر کلاس دو متد برای محاسبه ی مقدار تئوری میانگین و واریانس در نظر گرفتم. برای محاسبه میانگین و واریانس و همچنین تولید نمونه نیز از فرمول های معرفی شده از توضیحات پروژه استفاده کردم.

برای توزیع نرمال نیز یک staticmethod تعریف کردم تا با استفاده از توزیع نرمال استاندارد احتمال تجمعی را محاسبه کند تا در فاز های دیگر پروژه از آن استفاده کنم.

در تابع سازنده ی هر کدام از توزیع ها نیز بررسی شده تا پارامتر های ورودی با منطق ریاضیاتی در تناقض نباشند و اگر احیاناً تناقضی وجود داشته باشد، value error مناسب raise می شود تا ورودی اصلاح شود.

analysis.py:

در این ماژول توابع مختلفی برای آنالیز نتایج حاصل از توزیع ها تعریف شده است؛ یک تابع برای M بار استفاده از یک توزیع و تولید نمونه، دو تا تابع برای محاسبه واریانس و میانگین به صورت تجربی، یک تابع برای محاسبه درصد خطا و یک تابع برای ایجاد گزارش از محاسبات صورت گرفته.

ابتدای استفاده از این فاز، کاربر باید موتور تولید عدد را انتخاب کند و پس از آن توزیع مورد نظر را انتخاب کند و پارامتر های ورودی را به درستی وارد کند تا گزارش را دریافت نماید.

```
Choosing a random number generator:
1. LCG (Linear Congruential Generator)
2. XorShift
```

```
-----
Your Choice: 2
```

```
Choosing a distribution:
```

```
1. Bernoulli
2. Binomial
3. Geometric
4. Normal
5. Poisson
6. Exponential
```

```
-----
Your Choice: 1
```

```
Enter the probability of success(p): 2
The probability must be between 0 and 1.
Enter the probability of success(p):
```

```
Enter the probability of success(p): 0.5
Enter the number of times the sample will be generated(must be greater than 1): 100
```

```
Distribution name: Bernoulli
Params: P = 0.5
M: 100
Mean empirical: 0.54
Variance empirical: 0.2509090909090911
Mean theoretical: 0.5
Variance theoretical: 0.25
Error mean percent: 8.000000000000007
Error variance percent: 0.363636363636432

Press ENTER to return...
```

تصاویری از محیط برنامه در ترمینال: نحوه کار با فاز 1 و 2 و بررسی درستی پارامتر های ورودی

proof_of_theorems.py:

این ماژول برای فاز 3 پروژه طراحی شده و شامل توابعی است که اثبات کننده برخی قضایای احتمال هستند.

دو تابع اول در این ماژول برای بررسی خاصیت بی حافظگی هستند؛ با این دو تابع می‌خواهیم اثبات کنیم که توزیع هندسی و توزیع نمایی دارای خاصیت بی حافظگی هستند.

در تابع اول مقادیر احتمال مورد نیاز را حساب می‌کنم و در تابع دوم گزارش این محاسبات را برمیگردانم.

$$P(X > s+t | X > s) = P(X > t)$$

برای محاسبه احتمال شرطی از فرمول $\frac{n(A \cap B)}{n(B)}$ استفاده کردم.

کاربر باید موتور تولید کننده عدد را انتخاب کند و سپس انتخاب کند که می‌خواهد برای کدام توزیع خاصیت بی‌حافظگی را بررسی کند؛ سپس باید پارامتر های لازم برای توزیع مورد نظر را وارد نماید و بعد از تعداد دفعات انجام آزمایش را وارد کند تا گزارش حاصل از این بررسی برایش به نمایش گذاشته شود.

Select the desired distribution:

1. Geometric
2. Exponential

Your Choice: 1

$P(X > s+t \mid X > s) = 0.4782608695652174$
 $P(X > t) = 0.48$
Difference = 0.0017391304347825765

Press ENTER to return...

Enter the probability of success(p): 0.5

Enter the number of times the sample will be generated (must be greater than 1): 100

Enter s: 2

Enter t: 1

در ادامه‌ی این ماژول توابعی طراحی شده که برای تقریب توزیع دوجمله‌ای توسط توزیع نرمال و تقریب توزیع پواسن توسط توزیع نرمال استفاده می‌شوند.

با تابع اول این بخش، احتمال را به صورت تجربی و از طریق شمارش حساب میکنم. با تابع دوم احتمال همان بازه را از روی توزیع نرمال حساب میکنم و در نهایت با staticmethod موجود در کلاس توزیع نرمال حاصل احتمالات را محاسبه کرده و تفاضل آنها را برمیگردانم که معادل $P(a \leq X \leq b)$ است.

در نهایت دو تابع برای تقریب دوجمله‌ای و پواسن طراحی کردم که ابتدا به صورت تجربی حساب میکند و پس از آن با فرمول‌های مربوط به هر توزیع، پارامترهای لازم برای توزیع نرمال را محاسبه میکنم و احتمال را با توزیع نرمال نیز تقریب میزنم. در نهایت هم گزارش این محاسبات را برمیگردانم.

Choose the desired limit theorem:

1. Binomial approximation by normal($n \geq 50$)
2. Poisson approximation by normal($\lambda \geq 30$)

Your Choice: 1

Probability Empirical = 1.0
Probability Normal = 0.999999999978906
Difference = 2.1094237467877974e-12

Press ENTER to return...

Enter the number of times to repeat the experiment($n \geq 50$): 50

Enter the probability of success(p): 0.5

Enter the number of times the sample will be generated(must be greater than 1)(M): 100

Enter the lower limit: 1

Enter the upper limit: 1000

در طراحی کلاس ها و توابع ماژول های مختلف این برنامه از فرمول های کلاسیک مربوط به هر بخش استفاده کردم که با توجه به توضیحات پروژه و جستجو در منابع مربوطه برخی از فرمول های ناآشنا را نیز پیدا کردم. در هر بخش با انتخاب موتور تولید عدد و انتخاب توزیع مورد نظر با ورودی دادن پارامتر های لازم، گزارش کاملی از نتایج بررسی ها و محاسبات نمایش داده می شود. برای هر بخش نیز اعتبار سنجی مناسب برای ورودی ها انجام میشود تا با منطق ریاضی مغایرت نداشته باشد و منجر به نتایج صحیح و قابل استناد شود.

منابع:

- [تبدیل باکس-مولر](#)
- [الگوریتم 32 بیتی XorShift](#)
- [توضیح روش تبدیل معکوس](#)

جهت دسترسی به فایل setup و فایل نصبی پروژه به این [لینک](#) مراجعه نمایید.